

MEDIATEK

MT8676 Android Secure Boot User Manual

版本: 1.0
出版日期: 2024-09-19

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-09-19	张萌	正式版

目录

版本记录 2

目录 3

图片目录 3

表格目录 3

1 Secure Boot 4

 1.1 概述 4

 1.1.1 名词解释 4

 1.1.2 BootRom 4

 1.2 安全启动 5

 1.2.1 安全启动检查流程 5

 1.3 下载代理认证 (DAA) 7

 1.4 安全特性配置 7

 1.4.1 生成密钥对 7

 1.4.2 开启安全启动 9

 1.4.3 编译软件 9

 1.4.4 签名 preloader 10

 1.4.5 签名其他镜像 10

 1.5 签名 DA 11

 1.6 efuse.xml 配置 12

 1.7 Authfile 生成 12

附件一 附加条款 13

图片目录

图 1-1. 安全启动检查流程 6

图 1-2. dakey.h 8

图 1-3. resignda 中的 DA 密钥 11

表格目录

表 1-1. 名词解释 4

1 Secure Boot

1.1 概述

本文档旨在提供 MT8676 SoC 及其配套 SDK 中安全启动功能的概览。在该产品的设计和部署周期中，功能和特性可能会发生变化，这些变化将在本文档的后续版本中记录。此外，本文档还介绍了如何启用安全特性。

1.1.1 名词解释

表 1-1. 名词解释

缩略词	全称及释义
USBDL	USB Download USB 下载
DA	Download Agent 下载代理 (包括 DA_BR 和 DA_PL)
DAA	Download Agent Authentication 下载代理认证
DRAM	Data Random Access Memory 动态随机存取内存
eFuse	Electronic Fuse 电子保险丝
eMMC	Embedded MultiMedia Card 嵌入式多媒体卡
EVB	Evaluation Board 开发板
NVM	Non-Volatile Memory 非易失性内存
ROM	Read-only Memory 只读存储器
SBC	Secure Boot Check 安全启动检查
SDK	Software Development Kit 软件开发包
SRAM	Static Random Access Memory 静态随机存取存储器
TEE	Trusted Execution Environment 可信执行环境
UART	Universal Asynchronous Receiver/Transmitter 通用异步收发器
USB	Universal Serial Bus 通用串行总线
USBDL	USB Download USB 下载

1.1.2 BootRom

BROM (BootROM) 是 SoC ROM 内的软件，无法修改。它是应用处理器执行的第一个软件。主要工作总结如下：

- 对 SoC 进行基础的硬件配置（例如设置 PLL，时钟等），以便启动系统。
- 从启动设备 UFS/eMMC/NAND/...）加载第一阶段引导程序以启动系统。
- 与主机 PC 端的工具通信，以加载下载代理（DA）软件，用于为没有软件或软件损坏的外部 Flash 进行镜像下载。
- 使用多个安全启动检查密钥，对第一阶段引导程序或下载代理进行安全启动检查。

1.2 安全启动

1.2.1 安全启动检查流程

安全启动检查流程用于从硬件信任根创建可信执行链。可以通过烧录 eFuse 中的 SBC_EN 来启用它。每次系统从上电复位中恢复时都会执行此流程。图 1-1 展示了基本的安全启动检查流程。详细描述如下。

1. 上电复位后，BROM 使用以下流程来验证非易失性存储器（NVM，例如 eMMC 或 NAND）中的安全启动检查（SBC）公钥（SBC_PUBK）。BROM 依次使用来自 eFuse 的公钥哈希（SBC_PUBK_HASH）。
 - (1) 从 eFuse 读取安全启动检查公钥（SBC_PUBK）的哈希值（SBC_PUBK_HASH）。
 - (2) 从 NVM 读取 SBC 公钥（SBC_PUBK）。
 - (3) 计算来自 (2) 的数据的哈希值。
 - (4) 检查来自 (1) 和 (3) 的数据是否相同。
2. BROM 加载并验证 Preloader，这是第一阶段的引导加载程序。首先，BROM 将 Preloader 从 NVM 读取到 SoC 的 SRAM 中，并使用安全启动检查公钥（SBC_PUBK）来认证 Preloader。如果 Preloader 验证成功，将执行 Preloader。验证方法使用 SHA256 计算哈希值，RSA（2048 位）和 MTK 或 PSS 填充（由第一阶段引导加载程序的头部决定）来进行验证。

注意：

 - 如果外部存储器是 NAND 闪存，BROM 支持第二份（副本）Preloader。当 BROM 无法加载第一份（副本）Preloader 时，BROM 会尝试加载/认证第二份（副本）Preloader。
3. 在第二阶段引导加载程序之后，引入了证书链，用于验证第二阶段引导加载程序、Linux 内核和其他镜像。证书链设计用于密钥管理委托，每个镜像可以使用不同的密钥对。每个镜像都附加了两个证书，CERT1 和 CERT2，它们都是 x509v3 格式的。涉及两个密钥对，根密钥对和镜像密钥对。CERT1 由根私钥签名，CERT2 由镜像私钥签名。因此，验证步骤将是在用公钥（SBC_PUBK）验证了 CERT1 之后，从 CERT1 中提取的镜像公钥用于验证

CERT2。需要注意的是，每个镜像的 CERT1 都由相同的根私钥签名，但每个镜像的 CERT2 可能由相同的镜像私钥签名，也可能不是。

4. Preloader 从 NVM 把第二阶段引导加载程序加载到非安全的 DRAM 区域并验证。它使用嵌入在 Preloader 中的公钥来认证第二阶段引导加载程序的 CERT1，并使用从 CERT1 中提取的镜像公钥来认证第二阶段引导加载程序的 CERT2。如果第二阶段引导加载程序验证成功，在 TEE 初始化后将执行第二阶段引导加载程序。验证方法使用 SHA256 计算哈希值，RSA（2048 位）和 MTK 或 PSS 填充进行验证。
5. Preloader 还从 NVM 加载 TEE 到安全的 DRAM 区域并验证。它使用第一阶段引导加载程序内的公钥来认证 TEE 镜像的 CERT1，并使用从 CERT1 中提取的镜像公钥来认证 TEE 镜像的 CERT2。如果 TEE 验证成功，TEE 将被执行，然后跳转到第二阶段引导加载程序。验证方法使用 SHA256 计算哈希值，RSA（2048 位）和 MTK 或 PSS 填充进行验证。
6. 第二阶段引导加载程序也使用嵌入其中的公钥和镜像公钥分别验证 Linux 内核和其他在此阶段需要验证的镜像的 CERT1 和 CERT2。验证方法使用 SHA256 计算哈希值，RSA（2048 位）MTK 或 PSS 填充进行验证。
7. 在 Linux 系统执行后，可以通过 Linux 内核中的 dm-verity（在块设备上）来验证只读镜像。

注意：

- 目前 dm-verity 仅支持 Linux 中的块设备（eMMC 上的 EXT4）。

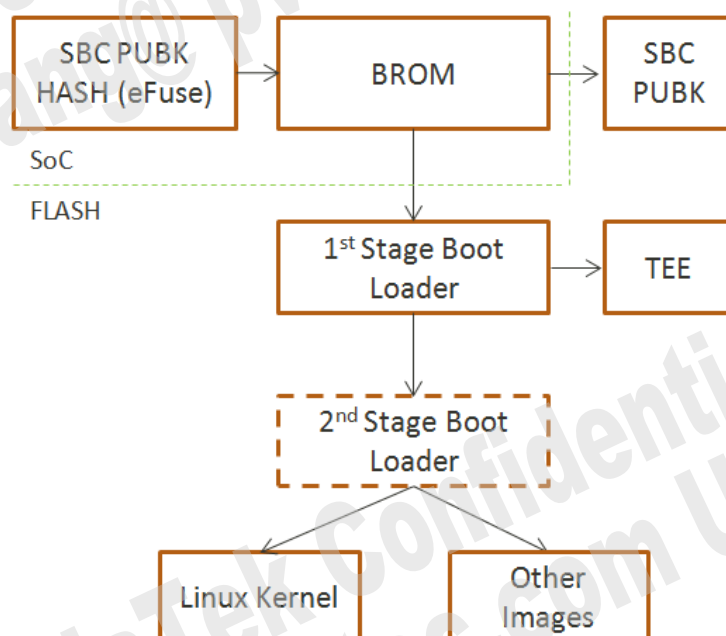


图 1-1. 安全启动检查流程

1.3 下载代理认证 (DAA)

当外部存储器 (eMMC/NAND/...) 中的设备软件为空或损坏时, 主机 PC 工具可以与 SoC 中的 BROM 通信, 将一个 DA 软件加载到 SoC 的 SRAM 中并执行 DA 来进行镜像下载过程。当 Enable_DAA 被烧录时, DA 将由 BROM 进行认证, 下面的流程被称为下载代理认证 (DAA)。

1. 主机 PC 工具通过 USB 或 UART 将认证文件 (AuthFile) 发送给 BROM。
2. BROM 对 AuthFile 进行认证。
 - (1) BROM 从 AuthFile 中读取安全启动检查 (SBC) 公钥 (SBC_PUBK), 并使用公钥哈希 (SBC_PUBK_HASH~SBC_PUBK_HASH1) 及相应的禁用位 (SBC_PUBK_HASH_DIS~SBC_PUBK_HASH1_DIS) 来认证 SBC 密钥的哈希。
 - (2) BROM 使用 SBC_PUBK 来验证 AuthFile。
 - (3) BROM 从 AuthFile 中获取 DAA 密钥。
3. 主机 PC 工具发送 DA。
4. BROM 使用 DAA 密钥通过 SHA256/RSA-2048 验证 DA。
5. BROM 跳转到 DA 来执行固件下载过程。

注意:

- SBC_PUBK 和 DAA 密钥可以不同, 以提高安全级别。
- 除了 BROM, 第一阶段引导加载程序也支持下载代理协议, 它可以使用 SHA256/RSA2048 对 DA 进行验证。

1.4 安全特性配置

1.4.1 生成密钥对

生成一对根密钥 (包括私钥和公钥)。使用 `pem_to_der.py`, 在

`vendor/mediatek/proprietary/scripts/sign-image_v2/der_extractor/` 中将密钥的格式转换为 DER 格式。

1. 生成私钥的命令:

```
openssl genrsa -out root_prvk.pem 2048
python pem_to_der.py root_prvk.pem root_prvk.der
```

2. 生成公钥的命令:

```
openssl rsa -in root_prvk.pem -pubout > root_pubk.pem
python pem_to_der.py root_pubk.pem root_pubk.der
```

使用相同的方法生成用于签名和验证镜像的镜像密钥对 (img_prvk.pem/img_pubk.pem) , 以及用于签名和验证 DA 的 DA 密钥对 (da_prvk.pem/da_pubk.pem) 。

1.4.1.1 生成 oemkey.h

将根密钥 (root_pubk.der) 导出, 使用 der_extractor 生成 oemkey.h, 该工具位于 vendor/mediatek/proprietary/scripts/sign-image_v2/der_extractor/ 目录下。请将 oemkey.h 放置到以下路径:

[DA] \$DA_Kit/Raphael-da/custom/\$PLATFORM/oemkey.h

[PL] \$PL/custom/\$PROJECT/inc/oemkey.h

[LK] \$LK/target/\$PROJECT/inc/oemkey.h

命令:

```
chmod 777 der_extractor
./der_extractor root_pubk.der oemkey.h ANDROID_SBC
```

1.4.1.2 生成 dakey.h

dakey.h 包含用于由 preloader 验证 DA_PL.bin 的 DA_PL 公钥。并使用相应的私钥对 DA_PL.bin 进行签名。

命令:

```
chmod 777 der_extractor
./der_extractor da_pubk.der dakey.h ANDROID_SBC
```

将 dakey.h 放到以下路径:

[PL] \$PL/custom/\$PROJECT/inc/dakey.h

注意:

- 生成 dakey.h 后, 请将“OEM”替换成“DA” (图 1-2).

```
#ifndef __DAKEY__
#define __DAKEY__
#define DA_PUBK_SZ 256
#define DA_PUBK {0xD1, 0x64, 0x03, 0x46, 0x6C, 0x
0xE3, 0x32, 0xE1, 0x7D, 0xC0, 0
0x4E, 0xAA, 0xC2, 0xCB, 0x30, 0
0x68, 0x20, 0x0D, 0xE1, 0x7D, 0
0x73, 0x70, 0x50, 0xF0, 0x84, 0
0x0B, 0xB0, 0xCD, 0x60, 0x68, 0
0x71, 0x5F, 0xEB, 0xDB, 0xF7, 0
```

图 1-2. dakey.h

1.4.2 开启安全启动

1.4.2.1 preloader 设置

1. vendor/mediate/proprietary/bootable/bootloader/preloader/custom/<project>/<project.mk>
MTK_SECURITY_SW_SUPPORT=yes
2. vendor/mediate/proprietary/bootable/bootloader/preloader/custom/<project>/<project.mk>
MTK_SEC_BOOT= ATTR_SBOOT_ONLY_ENABLE_ON_SCHIP
MTK_SEC_USBDL = ATTR_SUSBDL_ONLY_ENABLE_ON_SCHIP

ATTR_SBOOT_ONLY_ENABLE_ON_SCHIP 表示安全启动和安全下载是根据 eFuse SBC 字段来启用的。

然而，在项目的早期阶段，您可能不希望对设备进行 eFuse 操作。因此，可以在不使用 eFuse 的情况下强制启用安全启动和安全下载，以便首先验证软件的安全启动流程。

```
MTK_SEC_BOOT= ATTR_SBOOT_ENABLE  
MTK_SEC_USBDL = ATTR_SUSBDL_ENABLE
```

1.4.2.2 LK 设定

```
vendor/mediate/proprietary/bootable/bootloader/lk2/project/<project.mk>  
MTK_SECURITY_SW_SUPPORT=yes
```

1.4.2.3 Kernel 设定

32 bit kernel

```
<kernel path>/arch/arm/configs/<project>_debug_defconfig  
<kernel path>/arch/arm/configs/<project>_defconfig
```

64 bit kernel

```
<kernel path>/arch/arm64/configs/<project>_debug_defconfig  
<kernel path>/arch/arm64/configs/<project>_defconfig
```

```
CONFIG_MTK_SECURITY_SW_SUPPORT=y
```

1.4.3 编译软件

编译整个项目。

1.4.4 签名 preloader

1. 将您的 root_prvk.pem 替换到以下路径:

```
/vendor/mediatek/proprietary/bootable/bootloader/preloader/custom/${project}/security/chip_config/s/key/
```

2. 清除编译 preloader。

1.4.5 签名其他镜像

1.4.5.1 生成 cert1 和 cert2 密钥

1. 使用 root_prvk.pem 和 img_prvk.pem 生成 cert1 和 cert2_key。
2. 运行以下命令。

```
python ./vendor/mediatek/proprietary/scripts/sign-image_v2/img_key_deploy.py mt6897  
cert1_key_path=${KEY_PATH}/root_prvk.pem cert2_key_path=${KEY_PATH}/img_prvk.pem  
root_key_padding=pss 2>&1 | tee SecureGen.log
```

注意:

- 请不要进入 ./vendor/mediatek/proprietary/scripts/sign-image_v2/ 目录执行 img_key_deploy.py。
请在代码库的根目录下执行该命令。
- 请检查所有镜像的 cert1 和 cert2_key 是否已在以下位置更新,
vendor\mediatek\proprietary\custom\mt6897\security\cert_config\cert1\
vendor\mediatek\proprietary\custom\mt6897\security\cert_config\cert2_key\

1.4.5.2 签名镜像

在生成了 cert1 和 cert2_key 之后, 您可以运行签名脚本来生成 <image>-verified.bin 或 <image>-verified.img。

命令:

```
python ./vendor/mediatek/proprietary/scripts/sign-image_v2/SignFlow.py <platform> <project>  
2>&1 | tee signflow.log
```

或者

```
./vendor/mediatek/proprietary/scripts/sign-image/sign_image.sh 2>&1 | tee signflow.log
```

(必须在编译前 lunch project)

1.5 签名 DA

在签名 DA 之前，请确保已根据 1.4.1.1 章节配置了 oemkey.h。下载代理的引导区（DA_BR）和预加载程序（DA_PL）都使用 Python 脚本来进行签名过程。

1. 密钥配置 (da_prvk.pem 和 epp_prvk.pem 使用 RSA2048)

将 da_prvk.pem 和 epp_prvk.pem 放在 (图 1-3)

/vendor/mediatek/proprietary/scripts/secure_chip_tools/custom_keys/路径下。

- DA_PL: da_prvk.pem 应该与 dakey.h 中设置的公钥配对。
- DA_BR: da_prvk.pem 应该与 authfile 中的 DAA 公钥配对。

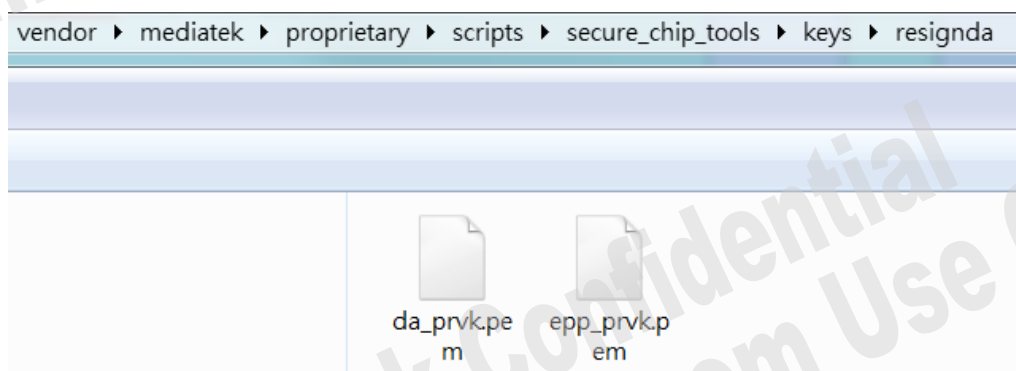


图 1-3. resignda 中的 DA 密钥

注意:

- epp_prvk 等同于 da_prvk.pem。因此，只需复制 da_prvk.pem 并重命名为 epp_prvk.pem。
- DA_PL 和 DA_BR 都使用 PSS 填充格式，所以如果 DA_PL 和 DA_BR 使用相同的密钥进行签名，预加载程序可以使用 DA_BR。如果使用不同的密钥对 DA_PL 和 DA_BR 进行签名，预加载程序只能使用 DA_PL。

2. 密钥路径设置

通常默认已经配置好了。

xref: /vendor/mediatek/proprietary/scripts/secure_chip_tools/settings/path_manager.py

Home | History | Annotate | Line# | Scopes# | Navigate# | Raw | Download

```

14 *****
15
16 # Root Key For Preloader, x509 Img, RMA Key Cert, and Auth File Verification
17 ROOT_PRVK      = "custom_keys/root_prvk.pem"
18 ROOT_PUBK      = "custom_keys/root_pubk.pem"
19
20 # Img Key For Preloader And x509 Img Content Cert Verification
21 IMG_PRVK       = "custom_keys/img_prvk.pem"
22 IMG_PUBK       = "custom_keys/img_pubk.pem"
23
24 # Key For DA Verification
25 DA_PRVK        = "custom_keys/da_prvk.pem"
26 EPP_PRVK       = "custom_keys/epp_prvk.pem"
27

```

Set the key path

3. 将需要签名的 DA 放置在 prebuilt/resignda/ 目录下，并执行以下命令来签名 DA：

```
python MTK/resign_da.py prebuilt/resignda/DA_BR.bin MT6897
settings/Legacy/da/bbchips_pss.ini all out/resignda/DA_BR-resing.bin
```

签名后的 DA 将生成在 out/resignda 目录下。

1.6 efuse.xml 配置

请向 MediaTek 申请 efuse.xml 文件。

- 在 efuse.xml 中设置密钥类型为 pss 并将 pub-key-e 设置为 010001。pub-key-n 字段应与 oemkey.h 中的根公钥相同。

```
<sbk-pub-key>
  <key-type>pss</key-type>
  <pub-key-e>010001</pub-key-e>
  <pub-key-n>
    9BB734774443D77557A76E24B107
  </pub-key-n>
```

- 将 SBC 设为 true 来开启安全启动功能。

```
Enable_SBC="true"
```

- 将 DAA 设为 true 来开启 DA 校验功能。

```
Enable_DAA="true"
```

1.7 Authfile 生成

DA 的公钥包含在 authfile 中，BROM 使用它来验证 DA。因此，如果启用了 DAA，在使用 flashtool 下载镜像时需要 authfile。所有位于以下位置的 .ini 文件，已经为本项目配置好了。

```
./vendor/mediatek/proprietary/scripts/secure_chip_tools/settings/Legacy/authfile/
```

您只需要替换位于以下位置的 DA、epp 和根私钥的 .pem 文件。

```
./vendor/mediatek/proprietary/scripts/secure_chip_tools/custom_keys/
```

Authfile 生成命令：

```
python MTK/toolauth.py -i settings/Legacy/authfile/toolauth_key.ini -g
settings/Legacy/authfile/toolauth_gfh_config_pss.ini out/toolauth/auth_sv5.auth
```

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。